Crisis Intelligence Core — Technical Engineering Specification

Correlation · Root-Cause · National Risk Index Cascade

Jordan Crisis Management Simulation Engine — Developer Specification v1.0 — 2026-05-31

This document specifies the **business logic as algorithms** for the engineers who will build the Crisis Intelligence Core: data structures, graph-traversal rules, scoring functions, pseudocode, and one worked scenario — the **Zarqa water-pipe cascade** — threaded through every engine. It answers three questions concretely: **(1)** how signals reported by different ministries are *stitched into one incident* instead of three unrelated tickets (§2–§3); **(2)** how the system finds the *root cause* — "why did the pipe rupture / pipeline explode?" — and separates it from the loud symptoms (§4); and **(3)** how the *National Risk Index* cascades through the dependency graph (§5). §6 binds them with contracts and runnable acceptance tests.

---

0. Canonical Crisis Graph & Reference Scenario

> **Audience:** engineers building the Crisis Intelligence Core. This document specifies *business logic as algorithms* — data structures, traversal rules, scoring functions, pseudocode, and worked numeric examples. It is intentionally **not** a business/strategy document.

Every engine in this spec (correlation, root-cause, risk) operates over a single shared structure: the **Crisis Dependency Graph (CDG)**. The graph is the substrate that lets the platform recognise that a pumping-station failure reported by the Ministry of Water and a hospital overload reported by the Ministry of Health are *the same crisis* and not three unrelated incidents.

0.1 Node types

Node	Key fields
`Signal`	`id, source_agency, type, observes_ref, location_ref, event_time, ingest_time, severity_raw, payload, freshness_s, src_confidence, provenance`
`Asset`	`id, class(pipe|pump|reservoir|hospital|junction|call_center…), attrs{}, location_ref, telemetry[]`
`Service`	`id, class(water_distribution|emergency_care|road_mobility|emergency_comms), provided_by[asset…], serves[pop…], criticality`
`Location`	`id, admin_code, geohash, name, level(site|district|governorate|national), parent_ref`
`Agency`	`id, name, domain`
`PopulationSegment`	`id, location_ref, size`
`Incident`	`id, signal_ids[], primary_location, span{t_start,t_end}, status, root_cause_ref?` (produced by §3)
`RootCause`	`id, cause_node_ref, mechanism, confidence, supporting[], conflicting[], missing[]` (produced by §4)
`RiskNode`	`id, subject_ref, level, score, factors{}, ts` (produced by §5)

0.2 Typed dependency edges

Each edge carries `w ∈ [0,1]` (a **propagation coefficient** = the fraction of an upstream disruption transferred to the downstream node) and `lag` (expected propagation delay).

`feeds(A→B)` · `supplies(A→B)` · `serves(Service→Pop)` · `depends_on(B→A, resource)` · `located_in(X→Location)` · `reported_by(Signal→Agency)` · `observes(Signal→Asset|Service|Location)` · `caused_by(Incident→Asset|Incident)` *(derived, §4)* · `correlated_with(Signal→Signal, score)` *(derived, §3)*

0.3 Reference scenario — the *Zarqa water-pipe cascade* (use these exact IDs in every worked example)

A single trunk-main rupture in Zarqa-North produces symptoms across four agencies. **Ground truth:** the rupture is the root cause; the 911 surge and hospital strain are the *highest-volume symptoms*, not the cause.

Location: `JO-AZ-N` (Zarqa Governorate, North district, `level=district`, `parent=JO-AZ`).

Assets / services:

- `PIPE-ZN-44` — trunk water main. `attrs{diameter_mm:600, install_year:1998, material:ductile_iron, last_inspection:overdue}`

- `PS-12` — pumping station (Agency `WAJ` / Ministry of Water). telemetry: `inlet_pressure_bar, outlet_flow_lps`
- `R-3` — reservoir
- `WATER-ZN` — Service `water_distribution`, `provided_by[R-3]`, `serves[POP-ZN]`, `criticality=0.9`
- `HOSP-ZN-1` — hospital (Agency `MoH`); provides `CARE-ZN`
- `DP-5` — emergency tanker distribution point (activated when `WATER-ZN` degrades)
- `JUNC-7` — road junction by `DP-5`; Service `ROAD-ZN`
- `PSAP-911` — national emergency call centre; Service `COMMS-911`
- `POP-ZN` — population segment, `size=180000`, `located_in JO-AZ-N`

Dependency edges `(w, lag)`:

```
PIPE-ZN-44 --feeds-->        PS-12        (w=0.95, lag=0)
PS-12      --supplies-->     R-3          (w=0.90, lag=5m)
R-3        --provides-->     WATER-ZN     (w=1.00, lag=5m)
WATER-ZN   --serves-->       POP-ZN       (w=1.00)
HOSP-ZN-1  --depends_on-->   WATER-ZN     (w=0.60, lag=30m)
DP-5       --activated_by--> WATER-ZN(deg)(w=0.70, lag=25m)
JUNC-7     --impacted_by-->  DP-5         (w=0.40, lag=40m)
COMMS-911  --load_from-->    POP-ZN       (w=0.50, lag=40m)
```

Raw signals (different agencies, different times): | ID | t | Agency | Observes | Content | severity_raw | |---|---|---|---|---|---| | `S1` | T+0 | WAJ SCADA | `PS-12` | inlet pressure 6.2→1.1 bar; outlet flow −85% | high | | `S2` | T+8m | WAJ | `WATER-ZN` | reservoir R-3 dropping; "distribution disruption Zarqa-North" | high | | `S3` | T+35m | MoH | `HOSP-ZN-1` | low water pressure, sanitation risk, intake rising | med | | `S4` | T+40m | Traffic/PSD | `ROAD-ZN` (JUNC-7) | congestion spike | low | | `S5` | T+45m | PSAP-911 | `COMMS-911` / `POP-ZN` | call volume Zarqa-North **+320%** ("no water","road blocked") | high | | `S6` | T+20m | Social | national | sentiment spike re: **fuel prices** — *unrelated; must NOT be stitched in* | low |

Intra-asset cause of the rupture (for §4 Layer B): a pressure transient on an aged (1998) ductile-iron main with overdue inspection (corrosion-thinned wall) → burst. The same mechanism logic generalises to a gas/oil *pipeline explosion* (failure modes: overpressure transient, corrosion/age, third-party strike).

---

1. Dependency-Graph Engine (the shared substrate)

Purpose: Provide the single in-memory property graph (the CDG of §0) plus the four traversal primitives that §3 (correlation), §4 (root-cause) and §5 (risk) call to reason about how a disruption propagates between assets and services.

1.1 Storage model

The CDG is a directed, weighted, typed property graph held as an adjacency map keyed by node id. One JSON schema covers every node type from §0.1; one covers every edge type from §0.2. Type-specific attributes live in `attrs{}`, so the storage layer stays uniform.

```
// Node
{
  "id": "PS-12",
  "type": "Asset",                    //
        Signal|Asset|Service|Location|Agency|PopulationSegment|Incident|RootCause|RiskNode
  "location_ref": "JO-AZ-N",
  "provenance": "live",               // live | sim:<runId>  — query-scoping tag (§1.5)
  "attrs": { "class": "pump", "inlet_pressure_bar": 1.1, "outlet_flow_lps": 12.0 }
}
// Edge (directed upstream→downstream in propagation order)
{
  "src": "PIPE-ZN-44",
  "dst": "PS-12",
  "rel": "feeds",                     //
       feeds|supplies|provides|serves|depends_on|activated_by|impacted_by|load_from|located_in|caused_by|correlated_with
  "w": 0.95,                          // propagation coefficient ∈[0,1]
  "lag_s": 0,                         // expected propagation delay, seconds
  "provenance": "live",
  "attrs": {}
}
```

R1 (MUST). Every edge MUST carry `w ∈ [0,1]` and `lag_s ≥ 0`. *Check:* loader rejects any edge failing `0 ≤ w ≤ 1 ∧ lag_s ≥ 0`.

1.2 Weight semantics

`w` is the **fraction of an upstream disruption transferred to the downstream node**. For a path `p = n₀ → n₁ → … → nₖ` the **path weight** is the product of its edge weights:

```
pathWeight(p) = ∏(i=1..k) w(n_{i-1} → n_i)      pathLag(p) = Σ lag_s(edges)
```

Products (not sums) because attenuation compounds: a 0.90 stage feeding a 0.60 stage transfers 0.54. Path weight is monotonically non-increasing along a path, which makes `minPathWeight` a sound prune bound (any extension only shrinks it) — see §1.3.

1.3 Traversal primitives

All four are best-first (max-heap on running path weight) over the adjacency map.

- `descendants(n, maxHops, minPathWeight)` → set of `{node, pathWeight, pathLag}` reachable downstream within `maxHops` whose path weight ≥ `minPathWeight`. Used by §5 for cascade propagation.
- `ancestors(n, maxHops, minPathWeight)` → same, traversing edges in reverse (upstream). Used by §4 to enumerate candidate causes of a symptom.
- `kShortestCausalPaths(symptom, candidate, K)` → top-K upstream paths from `symptom` back to `candidate`, ranked by **descending path weight** (strongest causal chain first). Used by §4 to score `caused_by`.
- `subgraphWithin(location, window)` → node/edge slice whose `location_ref` is `location` or a descendant of it (via `located_in`) and whose signals fall in time `window`. Used by §3 to bound correlation to one locality.

Cycle damping (one rule). A node MAY be revisited only if doing so strictly increases its best-known path weight; since every `w ≤ 1` a revisit can only decrease it, so cycles terminate in ≤ `maxHops` expansions. *Check:* `feeds`-cycle test graph halts.

R2 (MUST). `kShortestCausalPaths` MUST return paths in non-increasing `pathWeight` order. *Check:* output is sorted-descending.

Complexity. Best-first with a visited-best map is `O(E log V)` per call on the live CDG (a few hundred nodes per governorate), well within the real-time budget of §6.

```
def kShortestCausalPaths(symptom, candidate, K, maxHops, minPW, prov="live"):
    # Walk UPSTREAM (reverse edges) from symptom toward candidate.
    heap = [(-1.0, 0, [symptom])]          # (-pathWeight, lagSum, path) ; max-heap via negation
    out, bestAt = [], {}                    # bestAt: node -> best pathWeight seen
    while heap and len(out) < K:
        negPW, lag, path = heappop(heap)
        pw, node = -negPW, path[-1]
        if node == candidate:
            out.append({"path": path, "pathWeight": pw, "pathLag": lag}); continue
        if len(path) - 1 >= maxHops: continue
        for e in inEdges(node, prov):       # edges u --rel--> node ; u is upstream cause
            if e.src in path: continue                    # simple paths only
            npw = pw * e.w
            if npw < minPW: continue                      # prune: weight only shrinks
            if npw <= bestAt.get(e.src, 0.0): continue    # cycle-damping rule
            bestAt[e.src] = npw
            heappush(heap, (-npw, lag + e.lag_s, path + [e.src]))
    return out                              # already weight-sorted by heap order
```

1.4 Zarqa subgraph (weighted adjacency list)

Live CDG slice for `subgraphWithin(JO-AZ-N, window)`, edges as `dst (w, lag)`:

```
PIPE-ZN-44 → PS-12      (0.95, 0s)
PS-12      → R-3        (0.90, 300s)
R-3        → WATER-ZN   (1.00, 300s)
WATER-ZN   → POP-ZN     (1.00, 0s)        WATER-ZN → HOSP-ZN-1 (0.60, 1800s)
WATER-ZN   → DP-5       (0.70, 1500s)     DP-5     → JUNC-7    (0.40, 2400s)
POP-ZN     → COMMS-911  (0.50, 2400s)
```

Worked trace — `kShortestCausalPaths(HOSP-ZN-1, PIPE-ZN-44, K=2)` (the §4 query "is the hospital strain caused by the pipe?"). Walking upstream `HOSP-ZN-1 → WATER-ZN → R-3 → PS-12 → PIPE-ZN-44`:

step	edge (down→up)	w	running pathWeight	running pathLag
1	HOSP-ZN-1 ← WATER-ZN	0.60	0.60	1800s
2	WATER-ZN ← R-3	1.00	0.600	2100s
3	R-3 ← PS-12	0.90	0.540	2400s
4	PS-12 ← PIPE-ZN-44	0.95	**0.513**	2400s

Top path: weight **0.513**, lag 2400 s (40 m) — strong enough to support a `caused_by` edge in §4. The disjoint chain `S5/COMMS-911 ← POP-ZN ← WATER-ZN ← R-3 ← PS-12 ← PIPE-ZN-44` yields `0.50·1.00·1.00·0.90·0.95 = 0.4275`, which is below the hospital path's 0.513, ranking the 911 surge as a weaker (downstream-only) symptom path — exactly the §0 ground truth that the 911 spike is a high-volume *symptom*, not the cause. `S6` (national fuel sentiment) shares no edge into `JO-AZ-N`, so `subgraphWithin` never returns it and it is structurally un-stitchable (§3).

1.5 Provenance scoping

Every node and edge carries `provenance ∈ {live, sim:<runId>}`. Traversals take a `prov` argument; `inEdges/outEdges` MUST yield only edges whose provenance matches the requested scope, with `sim:<runId>` overlaying `live` (a sim run sees live topology plus its own injected edges, never another run's).

R3 (MUST). A traversal with `prov="live"` MUST NOT visit any `sim:*` node or edge. *Check:* inject a `sim:r1` burst on `PIPE-ZN-44`; a `live` `descendants(PIPE-ZN-44,…)` returns no `sim:r1` nodes; `descendants(…, prov="sim:r1")` does.

1.6 Required indexes

1. **`byId`** — `id → node` (O(1) node fetch; backs every traversal).

2. **`byLocation`** — `location_ref → {node ids}`, including transitive `located_in` closure (backs `subgraphWithin`, §3).
3. **`adjByProv`** — `(node id, direction, provenance) → edge list` (backs `inEdges` / `outEdges` under §1.5 scoping).

R4 (MUST). Edge insert/update MUST keep `byId`, `byLocation`, and `adjByProv` consistent in the same transaction. *Check:* post-insert, `outEdges(src)` contains the new edge and `byLocation` resolves both endpoints.

---

2. Signal Ingestion, Normalization & Entity Resolution

Purpose: turn heterogeneous agency reports into canonical `Signal` nodes bound to the *same* CDG entities, so that §3 correlation operates on shared foreign keys rather than free text.

This is the prerequisite for everything downstream. If `S1` (WAJ SCADA on `PS-12`) and `S5` (911 free-text "no water in Zarqa North") resolve to *different, unrelated* graph subtrees, §3 cannot stitch them and the platform reports three separate crises instead of one cascade. Entity resolution is what makes `PS-12 --supplies--> R-3 --provides--> WATER-ZN --serves--> POP-ZN` a single traversable path.

2.1 Pipeline & raw envelope

```
raw report → parse/validate → field-normalize → spatial-normalize
           → temporal-normalize → dedupe → resolve() → canonical Signal → CDG
```

Each agency POSTs a `RawReport`. The ingestor MUST reject (4xx) any report missing `source_agency`, `event_id`, or `event_time` (verifiable: schema validation unit test).

```
{ "source_agency":"WAJ", "event_id":"scada-9f2",
  "observed":{"kind":"asset","native_ref":"PS-12","class_hint":"pump"},
  "geo":{"lat":32.092,"lon":36.088,"admin_hint":"Zarqa-North","text":null},
  "event_time":"T+0", "ingest_time":"T+0:42",
  "severity_native":{"scale":"WAJ_PRESSURE","value":"high"},
  "payload":{"inlet_bar":[6.2,1.1],"outlet_flow_pct":-85} }
```

2.2 Field normalization

Per-agency severity scales are unified to `severity ∈ [0,1]` via a registry; enums/units are mapped to canonical SI.

```
SEV = {"WAJ_PRESSURE":{"low":0.25,"med":0.5,"high":0.85},
       "MoH_TRIAGE":  {"low":0.30,"med":0.55,"high":0.80},
       "PSAP_VOL":    {"low":0.20,"med":0.6,"high":0.90}}
def norm_fields(r):
    s = SEV[r.severity_native.scale][r.severity_native.value]   # → [0,1]
    return {"severity": s,
            "payload":  to_si(r.payload),       # bar, lps, %→ratio
            "src_confidence": AGENCY_TRUST[r.source_agency]}     # static prior
```

`S1 high → 0.85`, `S3 med → 0.55`, `S5 high → 0.90`. Every `scale` value MUST exist in `SEV` (check: registry-coverage test fails on unknown scale). Unmapped enums are quarantined, not silently defaulted.

2.3 Spatial normalization

- **Geohash:** encode `(lat,lon)` to **precision 7** (~153 m × 153 m) as the standard cell; `level=site` signals MAY use precision 8.
- **Admin resolution:** map `admin_hint` / geohash to a canonical `Location.admin_code` via point-in-polygon over admin boundaries; "Zarqa-North" → `JO-AZ-N`.
- **Point→asset snapping:** snap to the nearest same-class-compatible `Asset` within radius **R=250 m** (configurable per class). Beyond R, bind to the `Location` only, leaving `asset_ref=null` for §3 to associate by service.

```
def snap(geo, class_hint):
    loc = admin_lookup(geo)                          # → Location
    cand = assets_in_geohash(geo, p=7, expand=1)   # neighbour cells
    cand = [a for a in cand if compat(a.class, class_hint)]
    a = argmin(cand, key=lambda a: haversine(geo, a.location))
    return (a if a and haversine(geo,a.location) <= 250 else None), loc
```

Complexity `O(k)` in candidates per cell (geohash index lookup is `O(1)` amortized).

2.4 Temporal normalization & dedupe

- `event_time` (when it happened) drives all correlation windows in §3; `ingest_time` (when we got it) drives SLA/freshness only. `freshness_s = ingest_time − event_time` (`S1`: 42 s).
- **Buckets:** floor `event_time` to **60 s** buckets for windowing.
- **Out-of-order / late:** the stream MUST accept events up to `watermark_lag = 60 min` behind the watermark and re-open the affected bucket for §3 re-evaluation; later than that → `late_archive` (audited, not correlated). This covers `S6@T+20m` arriving after `S3@T+35m`.
- **Idempotent dedupe key** `= sha1(source_agency | event_id)`. Re-delivery MUST be a no-op upsert (check: posting `S1` twice yields one node). Distinct agencies reporting the same physical event get **distinct** Signals — they are merged

downstream by §3, never here.

2.5 Entity resolution — `resolve(signal)`

Resolution writes the three foreign keys §3 depends on: `asset_ref`, `service_ref`, `location_ref`. **Deterministic** path: if `native_ref` matches a known `(agency, native_id)` alias → bind directly (`S1.PS-12`). **Probabilistic** path: otherwise generate candidates and score.

Blocking key = `geohash_p5 ⊕ class_bucket` (cheap, recall-oriented). For each candidate entity `c`, compute features and a weighted score:

feature	def	weight
`f_geo`	`1 − min(1, dist_m/500)`	0.30
`f_name`	token-set ratio(text, c.name+aliases)	0.25
`f_class`	class compatibility 0/0.5/1	0.20
`f_time`	`1 − min(1, Δt/window)` vs active incidents	0.15
`f_serv`	shares Service with an active candidate	0.10

`score = Σ wᵢ·fᵢ`. Bind if `score ≥ 0.72`; `0.55 ≤ score < 0.72` → bind to `Location` + flag `needs_review`; `< 0.55` → `Location`-only.

```
def resolve(sig):
    if (a := alias_lookup(sig.source_agency, sig.native_ref)):
        return bind(sig, asset=a, service=service_of(a), loc=a.location)
    asset, loc = snap(sig.geo, sig.class_hint)
    if asset: return bind(sig, asset, service_of(asset), loc)
    best, sc = None, 0.0                          # probabilistic
    for c in candidates(block_key(sig)):
        s = (0.30*f_geo(sig,c)+0.25*f_name(sig,c)+0.20*f_class(sig,c)
           +0.15*f_time(sig,c)+0.10*f_serv(sig,c))
        if s > sc: best, sc = c, s
    if sc >= 0.72: return bind(sig, *expand(best))
    if sc >= 0.55: return bind(sig, service=svc_of(loc), loc=loc, review=True)
    return bind(sig, loc=loc)                     # location-only
```

Worked example — `S5` (911 free-text "no water, road blocked", Zarqa-North). No alias, no asset within R. Candidates from block key `JO-AZ-N`: `WATER-ZN`, `POP-ZN`, `ROAD-ZN`.

candidate	f_geo	f_name	f_class	f_time	f_serv	score
`WATER-ZN`	1.00	0.80 ("no water")	1.0	0.90	1.0	**0.30+0.20+0.20+0.135+0.10 = 0.935**
`POP-ZN`	1.00	0.40	0.5	0.90	1.0	0.735
`ROAD-ZN`	1.00	0.50 ("road blocked")	1.0	0.85	0.0	0.753

`S5` binds to `WATER-ZN` (0.935), and being a population-load signal also carries `location_ref=JO-AZ-N` / `service_ref=COMMS-911`. This co-location with `S1→S2` is exactly what lets §3 link the SCADA fault to the call surge.

2.6 Zarqa resolution table (§3 input)

Signal	path	asset_ref	service_ref	location_ref	severity
`S1`	deterministic alias	`PS-12`	`WATER-ZN`	`JO-AZ-N`	0.85
`S2`	alias (service)	`R-3`	`WATER-ZN`	`JO-AZ-N`	0.85
`S3`	alias	`HOSP-ZN-1`	`CARE-ZN`	`JO-AZ-N`	0.55
`S4`	snap→JUNC-7	`JUNC-7`	`ROAD-ZN`	`JO-AZ-N`	0.20
`S5`	probabilistic 0.935	—	`WATER-ZN` / `COMMS-911`	`JO-AZ-N`	0.90
`S6`	location-only	—	—	`JO` (national)	0.20

`S6` resolves to national scope with no Zarqa service binding — correctly isolating the unrelated fuel-price sentiment so §3 will **not** stitch it into the water incident. The five Zarqa signals share `location_ref=JO-AZ-N` and overlapping `service_ref`s, giving §3 the foreign-key overlap it needs to recognize one incident, not five. See §1 for the CDG node/edge store these refs index into, and §3 for how `correlated_with` edges are derived from them.

---

3. Event Correlation & Incident Stitching Engine

Purpose: decide which raw `Signal`s describe the *same* crisis, fusing them into one `Incident` over the CDG instead of N disconnected alerts.

The engine scores every pair of in-window signals across four dimensions, combines them into a link score `L`, builds a link graph, and emits the high-weight connected components as incidents. The decisive dimension is **CDG reachability** (§3.3): spatial/temporal coincidence is necessary but not sufficient — only a dependency path turns coincidence into causation.

3.1 Inputs and windowing

The engine consumes the normalized `Signal` stream from §2 (each carrying `observes_ref`, `location_ref`, `event_time`, `severity_raw→severity`, `src_confidence`). It runs on a **sliding window** keyed by `event_time`: window `W = [now − H, now]`, default horizon `H = 90m` (≥ longest cumulative CDG `lag`: COMMS-911 at 40m, DP-5→JUNC-7 at 25m+40m). Only signal pairs whose times fall within `H` are scored, bounding pairwise cost.

- The engine MUST window on `event_time`, not `ingest_time` (check: replay S5 with a 30m ingest delay → still stitched to INC-ZN-WATER).
- The engine MUST re-evaluate an open incident when any new in-window signal arrives (streaming merge/split, §3.6).

3.2 Dimension scores `dim_k ∈ [0,1]`

Each function takes a signal pair `(si, sj)` and returns `[0,1]`. Resolve each signal's anchor entity `e = observes_ref` and location `loc = location_ref`.

(1) Spatial `d_spatial`. Reward co-location on the CDG or admin/geographic proximity.

```
if same anchor asset/service OR e_i,e_j co-located_in same site   -> 1.0
elif loc_i == loc_j (same admin_code, e.g. JO-AZ-N)               -> 0.9
elif adjacent districts (share parent JO-AZ) OR geo_dist ≤ d=5km  -> 0.6
elif same governorate                                             -> 0.3
else                                                              -> 0.0
```

(2) Temporal `d_temporal`. Not mere closeness — **causal ordering**. Order `(up,down)` so `e_up` is the CDG-upstream entity (§3.3 path direction). Let `Δ = t_down − t_up`, `lag = Σ edge.lag` along the up→down CDG path, tolerance `τ = max(10m, 0.5·lag)`.

```
if no path between e_i,e_j        -> 0.0          # ordering undefined
expected = lag
if Δ < 0                          -> 0.0          # symptom precedes cause: reject
else: d_temporal = clamp(1 − |Δ − expected| / (expected + τ), 0, 1)
```

(3) Dependency-graph reachability `d_dep` — decisive. Run a bounded bidirectional search (≤ `MAXHOP=4`) over `supplies|feeds|provides|depends_on|serves|activated_by|impacted_by|load_from` edges between `e_i` and `e_j`. Path weight = product of edge `w`.

```
P = max-weight CDG path(e_i, e_j), len ≤ MAXHOP   # either direction
if none -> d_dep = 0.0
else    -> d_dep = Π_{edge∈P} edge.w
```

(4) Causal/semantic `d_sem`. Look up the ordered signal-type pair in a curated **cascade-pattern table**; value = pattern prior.

```
PATTERNS = {
 (water_supply_drop, sanitation_risk):    0.9,   # WATER-ZN -> HOSP-ZN-1
 (water_supply_drop, care_demand):        0.85,
 (service_degrade,   tanker_activation):  0.8,   # WATER-ZN -> DP-5
 (tanker_activation, road_congestion):    0.7,   # DP-5 -> JUNC-7
 (service_outage,    call_surge):         0.8,   # POP-ZN -> COMMS-911
 (pressure_loss,     supply_drop):        0.9 }  # PS-12 -> WATER-ZN
d_sem = PATTERNS.get((type_up, type_down), 0.2)  # 0.2 = weak generic prior
```

3.3 Pairwise link score

```
L(si,sj) = Σ_k θ_k · dim_k ,  with confidence gate on src_confidence
θ = { spatial:0.20, temporal:0.20, dep:0.45, semantic:0.15 }   # Σθ = 1
L_conf = L · min(src_confidence_i, src_confidence_j)
LINK if L_conf ≥ T_link (=0.55)  AND  d_dep > 0   # hard dependency gate
```

- **Hard dependency gate:** a pair with `d_dep = 0` MUST NOT link, regardless of spatial/temporal/semantic score (check: S6 vs any → no link). This is the primary over-merge control: it forbids stitching two entities the CDG does not connect.
- `θ_dep` dominates (0.45) so reachable pairs clear threshold while merely-nearby pairs do not.

3.4 Worked Zarqa pairwise matrix

Anchors/paths (from §0 edges): S1=`PS-12`, S2=`WATER-ZN`, S3=`HOSP-ZN-1` (depends_on WATER-ZN, w=0.60, lag=30m), S4=`JUNC-7` (impacted_by DP-5 w=0.40, lag=40m; DP-5 activated_by WATER-ZN w=0.70, lag=25m), S5=`COMMS-911` / `POP-ZN` (load_from POP-ZN w=0.50, lag=40m; POP served by WATER-ZN), S6=national fuel-price (no CDG anchor).

`d_dep` examples: S1–S2 path `PS-12→R-3(0.90)→WATER-ZN(1.00)` = **0.90**; S2–S3 = depends_on **0.60**; S2–S4 = `0.70·0.40` = **0.28**; S2–S5 = `1.00·0.50` = **0.50**. `d_temporal` S1–S2: Δ=8m, expected≈5m, τ=10m → `1−3/15` = **0.80**. S2–S3: Δ=27m, expected=30m, τ=15m → `1−3/45` = **0.93**.

Pair	d_sp	d_tmp	d_dep	d_sem	L	Rule firing / verdict
S1–S2	0.9	0.80	0.90	0.90	**0.86**	pressure→supply; **LINK**
S2–S3	0.9	0.93	0.60	0.90	**0.74**	supply→sanitation; **LINK**
S2–S5	0.9	0.70	0.50	0.80	**0.65**	outage→call-surge; **LINK**
S2–S4	0.6	0.60	0.28	0.70	**0.47**	weak path; no direct link*
S4–S5	0.9	0.55	0.20	0.20	**0.42**	co-located only; no link
S6–*	0.0	0.00	0.00	0.20	**0.03**	**REJECTED** (dep gate=0)

*S4 (`L=0.47<0.55`) does not link directly to S2, but DP-5→JUNC-7 is a real CDG edge; S4 joins via a **secondary path bonus**: any signal whose anchor is CDG-reachable (≤MAXHOP) from a current cluster member with cumulative `w ≥ 0.15` is admitted at reduced weight. S2→DP-5→JUNC-7 = `0.70·0.40 = 0.28 ≥ 0.15` → S4 admitted as peripheral member (`link_confidence=0.47`). S6 has no anchor and no path → stays out.

Result `INC-ZN-WATER`: members `{S1,S2,S3,S4,S5}`; `primary_location=JO-AZ-N` (modal `location_ref` of core members); `span = [min event_time, max event_time] = [T+0, T+45m]`; `status=open`.

3.5 Stitching algorithm

```
def stitch(W):                          # W: signals with event_time in horizon
  idx = spatial_index(W)                # geohash + admin_code buckets
  G = LinkGraph()
  for si in W:
    for sj in candidates(idx, si):      # only same/adjacent admin or ≤d km
      if abs(si.t − sj.t) > H: continue
      dims = score_dims(si, sj)         # §3.2; d_dep via bounded CDG search
      if dims.dep == 0: continue        # hard gate, prunes S6 cheaply
      L = theta · dims
      Lc = L * min(si.conf, sj.conf)
      if Lc >= T_link:
        G.add_edge(si, sj, weight=Lc)
  comps = connected_components(G)       # union-find
  incidents = []
  for C in comps:
    if weighted_density(C) < D_min:     # over-merge guard, split weak bridges
      C = louvain_split(C)              # min-cut on lowest-weight edges
    for sub in as_clusters(C):
      sub = admit_peripheral(sub, idx) # secondary-path bonus (S4)
      incidents.append(make_incident(sub))
  return reconcile(incidents, open_incidents())   # streaming merge/split, §3.6
```

Complexity. Spatial bucketing yields ≈ `k` candidates per signal (k ≪ |W|), so pairwise scoring is `O(|W|·k)` not `O(|W|²)`. Each `d_dep` search is `O(b^(MAXHOP/2))` with bidirectional pruning, `b` =avg CDG degree → effectively constant for `MAXHOP=4`. Union-find is near-linear. Overall **`O(|W|·k·b²)`** per tick.

3.6 Over-merge control, confidence, streaming merge/split

- **Over-merge guard.** Two genuine incidents sharing one weak bridge edge MUST be split if `weighted_density < D_min` (=0.4); the engine cuts the lowest-weight edge and re-checks (check: inject an unrelated reachable signal at `L=0.56` — it is severed, not merged).
- **Link confidence** propagates: `incident.confidence = 1 − Π(1 − Lc_e)` over spanning-tree edges; core S1–S2–S3 spine gives `≥0.9`.
- **Streaming reconcile.** On each tick `reconcile()` compares new components to open incidents by member-ID Jaccard: **overlap ≥ 0.5 → MERGE** (extend `span`, append `signal_ids`); a component that fragments below `D_min` → **SPLIT** (new incident IDs, both reference parent). Incidents idle `> 2·H` → `status=resolved`.
- An incident MUST carry every contributing signal id and MUST NOT include any signal failing the dependency gate (check: S6 absent from `INC-ZN-WATER.signal_ids`).

3.7 Incident schema


```
{
  "id": "INC-ZN-WATER",
  "signal_ids": ["S1","S2","S3","S4","S5"],
  "primary_location": "JO-AZ-N",
  "span": { "t_start": "T+0", "t_end": "T+45m" },
  "status": "open",
  "confidence": 0.93,
  "members": [
    { "signal_id":"S1", "anchor":"PS-12",      "role":"core",       "link_confidence":0.86 },
    { "signal_id":"S2", "anchor":"WATER-ZN",   "role":"core",       "link_confidence":0.86 },
    { "signal_id":"S3", "anchor":"HOSP-ZN-1",  "role":"core",       "link_confidence":0.74 },
    { "signal_id":"S5", "anchor":"COMMS-911",  "role":"core",       "link_confidence":0.65 },
    { "signal_id":"S4", "anchor":"JUNC-7",     "role":"peripheral", "link_confidence":0.47 }
  ],
  "link_graph": [["S1","S2",0.86],["S2","S3",0.74],["S2","S5",0.65]],
  "rejected": [{ "signal_id":"S6", "reason":"dependency_gate", "d_dep":0.0 }],
  "root_cause_ref": null
}
```


The emitted `Incident` (and its `signal_ids` / `link_graph`) is the sole input to the Root-Cause Analysis Engine (§4), which selects the causal apex (`PS-12` / `PIPE-ZN-44`) from these members; `primary_location` and `span` feed the Risk Index (§5). The CDG paths used here are the same edges traversed for cascade propagation in §5 and defined in §1.

---

4. Root-Cause Analysis Engine

Purpose: answer "*why did it happen / why did the pipeline explode?*" by traversing the CDG (§1) **backward** from an Incident's symptoms to the common upstream driver (Layer A), then explaining that driver's intra-asset failure mechanism (Layer B).

The engine consumes an `Incident` (produced by §3) and emits one `RootCause` record. The wicked-problem requirement is **symptom-vs-cause discrimination**: the loudest signals (`S5` 911 +320%, `S3` hospital strain) are *high-volume symptoms*; the true cause (`PS-12` / `PIPE-ZN-44`) is low-volume. The ranker MUST NOT confuse volume with causality.

4.1 Layer A — cross-symptom root cause

Input is the Incident's symptom set: each symptom is the asset/service a signal `observes` plus its `event_time` and `severity`. For Zarqa the Incident from §3 stitches `{S1,S2,S3,S4,S5}` (S6 excluded). We map each signal to its observed CDG node:

Signal	Observes node	event_time	role
`S1`	`PS-12`	T+0	near-source telemetry
`S2`	`WATER-ZN`	T+8m	mid-chain
`S3`	`HOSP-ZN-1` → `WATER-ZN`	T+35m	symptom
`S4`	`JUNC-7`	T+40m	symptom
`S5`	`COMMS-911` / `POP-ZN`	T+45m	symptom (loudest)

Candidate generation. For each symptom node we call `ancestors(symptom, maxHops, minPathWeight)` from §1 (the open-ended upstream enumeration), giving the candidate-cause set `C`; then for each candidate `c ∈ C` we call `kShortestCausalPaths(symptom, c, K=1)` to obtain the strongest cause→symptom chain used for scoring. For Zarqa, the ancestor enumeration from every symptom converges on the common ancestors `{PS-12, PIPE-ZN-44, R-3, WATER-ZN}`.

Scoring. Each candidate `c ∈ C` is scored against the symptom set `Σ`:

```
score(c) = w_cov·COV + w_path·PATH + w_temp·TEMP + w_corr·CORR
weights (default, MUST be config): w_cov=0.30, w_path=0.25, w_temp=0.20, w_corr=0.25
```

- **COV (coverage)** = (# symptoms reachable from `c` via a causal path) / |Σ|. Penalises shallow candidates that explain only the loud symptom.
- **PATH (path strength)** = mean over reached symptoms of the path weight `Π w_e` (product of edge `w` along the cause→symptom path). Long, lossy chains score lower.
- **TEMP (temporal consistency)** = fraction of reached symptoms whose observed lag `Δt = t_symptom − t_cause` matches the path's expected `Σ lag` within tolerance τ (default ±50%). A candidate that *post-dates* a symptom scores 0 on that symptom (effect cannot precede cause).
- **CORR (direct corroboration)** = normalized weight of signals that *directly observe* `c` or its telemetry (e.g. `S1` SCADA pressure transient on `PS-12`). A candidate with first-party instrument evidence is boosted; a candidate inferred only by traversal is not.

Symptom-vs-cause discrimination falls out of COV+PATH+CORR: `COMMS-911` reaches **only itself** (COV=0.2), has **no** ancestor instrumentation (CORR=0), and cannot temporally precede `S1` (TEMP penalised). `PIPE-ZN-44` / `PS-12` reach **all five** symptoms with high path products and carry `S1`'s direct SCADA corroboration. Volume never enters the formula.

Output. `likely_cause` = argmax; `hypotheses` = full ranked list (the engine MUST return ≥2 when $\text{score}_1 - \text{score}_2 < 0.15$, never forcing a single cause).

```
{ "id": "RC-...", "incident_ref": "INC-...",
  "likely_cause": "PIPE-ZN-44",
  "confidence": 0.80,
  "supporting_evidence": ["S1 (SCADA pressure transient+step)","S2 (reservoir drop)","path PIPE-ZN-44→…→HOSP-
        ZN-1 (0.513)"],
  "conflicting_evidence": ["S5-vol (911 +320% is the loudest signal yet resolves to a downstream symptom, not
        the cause)"],
  "missing_information": ["PIPE-ZN-44 acoustic/leak sensor (none deployed)"],
  "hypotheses": [
    {"cause":"PIPE-ZN-44","score":0.86,"covers":5},
    {"cause":"PS-12","score":0.81,"covers":5},
    {"cause":"WATER-ZN","score":0.54,"covers":3} ] }
```

```
def rankRootCauses(incident):
    Σ = [(sig.observes_node, sig.event_time, sig.severity)
         for sig in incident.signal_ids]
    C = set()
    paths = {}                          # (cause, symptom) -> path
    for (s, t_s, _) in Σ:
        for a in ancestors(s, maxHops=6, minPathWeight=0.05):   # §1: upstream enumeration
            c = a.node
            C.add(c)
            kp = kShortestCausalPaths(s, c, K=1)                # §1: strongest c→s chain
            if kp: paths[(c, s)] = kp[0]
    ranked = []
    for c in C:
        reached = [(s,t_s) for (s,t_s,_) in Σ if (c,s) in paths]
        COV  = len(reached)/len(Σ)
        PATH = mean(prod_edge_w(paths[(c,s)]) for (s,_) in reached) if reached else 0
        TEMP = mean(temporal_ok(paths[(c,s)], t_cause(c), t_s)   # 1/0 per symptom
                    for (s,t_s) in reached) if reached else 0
        CORR = corroboration(c, incident.signal_ids)  # S1 directly observes PS-12
        score = 0.30*COV + 0.25*PATH + 0.20*TEMP + 0.25*CORR
        ranked.append((c, score, len(reached)))
    ranked.sort(key=lambda r: -r[1])
    conf = confidence(ranked)           # §4.3
    return build_rootcause(incident, ranked, conf)

def temporal_ok(path, t_cause, t_symptom):
    expected = sum(e.lag for e in path.edges)
    observed = t_symptom - t_cause
    if observed < 0: return 0           # effect precedes cause -> impossible
    return 1 if abs(observed - expected) <= 0.5*max(expected, 1) else 0
```

Complexity: `O(|Σ| · K · (E·logV))` for the K-shortest-path fan-out (§1), then `O(|C|·|Σ|)` to score. **Edge cases:** (a) disjoint symptom clusters → return one hypothesis per cluster, flag *multi-cause*; (b) zero ancestors instrumented → CORR=0 for all, lower confidence + populate `missing_information`; (c) cycle in CDG → §1 guarantees acyclic causal-path enumeration.

Worked Zarqa trace (Layer A)

Causal path with cumulative weight (product of edge `w`) and expected lag:

```
PIPE-ZN-44 →feeds(.95)→ PS-12 →supplies(.90)→ R-3 →provides(1.0)→ WATER-ZN
  →depends_on(.60)→ HOSP-ZN-1   Πw=.95·.90·1·.60 = 0.513   Σlag=40m
  →activated_by(.70)→ DP-5 →impacted_by(.40)→ JUNC-7  Πw=.239  Σlag=75m
  →load_from(.50)→ COMMS-911 (via POP-ZN)            Πw=.428  Σlag=50m
```

Score matrix (arithmetic shown; `t_cause(PIPE)=T+0`):

Candidate	COV	PATH	TEMP	CORR	score
`PIPE-ZN-44`	5/5=1.0	mean(.95,.86,.51,.24,.43)=0.60	5/5=1.0	1.0 (S1 on child PS-12)	**0.30+0.150+0.20+0.25=0.86**
`PS-12`	5/5=1.0	mean(.90,.81,.54,.28,.45)=0.60	5/5=1.0	0.80	**0.30+0.150+0.20+0.20=0.81**
`WATER-ZN`	3/5=0.6	mean(.60,.27,.43)=0.43	3/3=1.0	0.20	**0.18+0.108+0.20+0.05=0.54**
`COMMS-911`	1/5=0.2	1.0 (self)	0/1 (cannot precede S1)	0.0	**0.06+0.250+0.00+0.00=0.31**

`PIPE-ZN-44` ranks **#1**; the loud 911 symptom `COMMS-911` ranks last. **Acceptance check passes.** `PS-12` is a close #2 (Δ=0.05<0.15) so both are returned as hypotheses, with Layer B disambiguating which physical asset failed.

4.2 Layer B — intra-asset failure causation

Given the implicated asset, infer *the physical mechanism* from `attrs` + `telemetry` against a **failure-mode knowledge base (FMKB)**. Each mode lists evidence predicates with weighted contributions; generalises from a water trunk-main burst to a gas/oil **pipeline explosion**.

```
{ "asset_class": "pipe",
  "modes": [
    {"id":"pressure_transient","priors":0.25,"evidence":[
       {"pred":"scada_pressure_step_drop","w":0.45},
       {"pred":"transient_spike_before_drop","w":0.35}]},
    {"id":"corrosion_age","priors":0.25,"evidence":[
       {"pred":"install_year < now-20","w":0.30},
       {"pred":"material in [ductile_iron,steel]","w":0.20},
       {"pred":"inspection_overdue","w":0.35}]},
    {"id":"third_party_strike","priors":0.25,"evidence":[
       {"pred":"excavation_permit_nearby","w":0.50},
       {"pred":"point_failure_no_precursor","w":0.30}]},
    {"id":"overpressure","priors":0.25,"evidence":[
       {"pred":"upstream_valve_event","w":0.50}]} ] }
```

```
def explainAssetFailure(asset):
    out = []
    for m in FMKB[asset.class].modes:
        s = m.priors
        for ev in m.evidence:
            if eval_pred(ev.pred, asset.attrs, asset.telemetry):
                s += ev.w
        out.append((m.id, s))
    Z = sum(s for _,s in out)
    ranked = sorted(((mid, s/Z) for mid,s in out), key=lambda x:-x[1])
    return {"mechanism": ranked[0][0],
            "confidence": ranked[0][1],
            "ranked_modes": ranked}
```

Worked Zarqa (asset `PIPE-ZN-44`): `install_year=1998` (<2006 ✓), `material=ductile_iron` ✓, `inspection=overdue` ✓; SCADA `S1` shows a transient spike then 6.2→1.1 bar step ✓✓.

Mode	priors + matched evidence	raw	normalized
corrosion_age	.25+.30+.20+.35	1.10	**0.42**
pressure_transient	.25+.45+.35	1.05	0.40
third_party_strike	.25 (no permit, but point failure +.30)	0.55	0.21 — *demoted; raw .85→see note*
overpressure	.25 (no valve event)	0.25	—

Mechanism = **corrosion/age-weakened wall failing under a pressure transient** (top two modes co-fire), confidence **0.42**, with `pressure_transient` as the proximate trigger. This is the wall-thinning-plus-water-hammer narrative §0 specifies. For a gas pipeline the same FMKB ranks `overpressure` / `third_party_strike` higher when a valve or excavation event is present.

4.3 Confidence semantics & requirements

`confidence ∈ [0,1] = score₁ · evidence_completeness`, where `evidence_completeness = |supporting| / (|supporting| + |missing|)` rewards first-party corroboration and penalises evidentiary gaps. A small margin to the #2 hypothesis does **not** depress confidence when the top hypotheses lie on the *same* causal chain (e.g. `PIPE-ZN-44` and `PS-12`); margin only governs whether ≥2 hypotheses are *returned* (above), it is not a confidence multiplier. Zarqa: `score₁=0.86`, supporting≈{S1 transient, S1 step, S2 drop} vs missing {pipe leak sensor} → `evidence_completeness ≈ 0.93` → **`confidence ≈ 0.80`**.

Requirements (RFC-2119):

- The ranker **MUST** rank the water-infra cause (`PIPE-ZN-44` / `PS-12`) strictly above every symptom node (`HOSP-ZN-1`, `JUNC-7`, `COMMS-911`). *Check:* assert `rank(PIPE-ZN-44) == 1` on the §0 fixture (see §6 acceptance test).
- A candidate that temporally **post-dates** a symptom **MUST** score `TEMP=0` for that symptom. *Check:* `temporal_ok` returns 0 when `observed<0`.
- The engine **MUST** record, as `conflicting_evidence`, any higher-volume symptom signal that competes with the chosen cause (e.g. `S5`'s 911 +320%), so the surfaced Insight (§6) shows the symptom-vs-cause tension it resolved. *Check:* `len(conflicting_evidence) ≥ 1` on the §0 fixture (the loud `S5`).
- The engine **MUST** return ≥2 hypotheses when `score₁−score₂ < 0.15`. *Check:* `len(hypotheses) ≥ 2` on the Zarqa fixture (Δ=0.05).
- It **MUST** populate `missing_information` whenever any implicated asset lacks first-party instrumentation. *Check:* `PIPE-ZN-44` (no leak sensor) appears in `missing_information`.
- It **SHOULD** propagate the chosen `RootCause.cause_node_ref` back as a `caused_by` edge (§0.2) for §5's cascade model. It **MAY** withhold a Layer-B mechanism (confidence below 0.30) and instead emit `missing_information` requesting an inspection.

---

5. National Risk Index & Cascade Propagation

Purpose: compute a deterministic, explainable National Risk Index over the CDG (§0) by scoring each node locally, forward-propagating impact along dependency edges, and rolling node→service→sector→governorate→national with attribution and anti-oscillation guards.

5.1 Index contract

The index is a pure function of the CDG snapshot, the active Incidents (§3) and RootCauses (§4). Fixed contract:

```
RiskIndex(level, subject) -> RiskNode{ score∈[0,100], factors{}, attribution[], ts }
INPUTS  (frozen):  base factors {sev, crit, expo, ready, fresh}, edge {w, lag}, decay δ, weights θ
RANGE   : every score, every aggregate, clamped to [0,100]
DETERMINISM: same snapshot ⇒ same RiskNode (no wall-clock except `ts`); MUST be reproducible.
```

`δ = 0.85` (per-hop decay), convergence ε = 0.5 idx-points, max 12 sweeps, cycle damping γ = 0.5.

5.2 Per-node base risk `r(n)`

Each scored node carries five normalized factors in `[0,1]`:

factor	symbol	source	normalization
signal severity	`sev`	max normalized `severity` of signals observing n (read from §2's per-agency registry; the scale at right is the default when an agency value is absent)	low=0.3, med=0.6, high=0.9, crit=1.0
service criticality	`crit`	`Service.criticality` (n or nearest service)	as-is
population exposure	`expo`	served `PopulationSegment.size`	`min(1, size/200000)`
response readiness	`ready`	tanker/crew availability, bed headroom	`1 − availability` (deficit)
data freshness/quality	`fresh`	`freshness_s`, `src_confidence` (§2)	`1 − src_confidence·e^(−freshness_s/3600)`

Weighted blend (weights `θ` sum to 1), scaled to 0–100:

```
r(n) = 100 · ( 0.30·sev + 0.20·crit + 0.20·expo + 0.15·ready + 0.15·fresh )
```

`r(n)` is the **local** risk before any inherited cascade impact. Final node score `R(n) = clamp(r(n) + impact(n), 0, 100)` where `impact(n)` comes from §5.3.

MUST: factors are clamped to `[0,1]` pre-blend; verify `0 ≤ r(n) ≤ 100` for any input.

5.3 Cascade propagation

Impact forward-propagates from a failed/disrupted node down dependency edges. Per edge `parent→child (w, lag)`:

```
impact(child) += impact(parent) · w(edge) · δ^hops
```

The graph is treated as a DAG; back-edges (cycles, e.g. COMMS load feeding readiness) are damped by `γ` and capped by the convergence loop so a cycle cannot inflate without bound. `lag` time-shifts when a child's impact *activates* — at evaluation time `t_eval`, an edge contributes only if `t_eval ≥ t_parent_onset + lag`. This makes the index a function of elapsed time since onset.

```
def propagateCascade(failedNode, t_eval):
    impact = defaultdict(float)
    impact[failedNode] = r(failedNode)          # seed = local base risk
    onset = {failedNode: failedNode.onset_t}
    for sweep in range(MAX_SWEEPS):              # iterate to convergence
        delta_max = 0
        for (p, c, w, lag) in edges_topo_order():   # topo where acyclic
            if t_eval < onset.get(p, INF) + lag:    # lag gate
                continue
            hops = hop_distance(failedNode, c)
            contrib = impact[p] * w * (DECAY ** hops)
            if is_back_edge(p, c):                  # cycle damping
                contrib *= GAMMA
            new = impact[c] + contrib
            delta_max = max(delta_max, abs(new - impact[c]))
            impact[c] = new
            onset.setdefault(c, onset[p] + lag)
        if delta_max < EPSILON: break
    for n in impact: impact[n] = min(impact[n], 100)  # per-node cap
    return impact   # attribution recorded per (p→c) contrib (see 5.5)
```

Complexity `O(SWEEPS · |E|)`. Edge cases: missing `lag` ⇒ treat as 0; node with no scored signal ⇒ `r(n)` uses `sev=0`; unrelated signals (S6) never touch the WATER subgraph because there is **no edge path** from the fuel-sentiment node to `WATER-ZN`, so its impact contribution is exactly 0.

5.4 Worked Zarqa example — PIPE-ZN-44 rupture

Base factors at `t_eval = T+45m` (after S1–S5 landed, S6 excluded). Seed: `r(PIPE-ZN-44)=90` (high sev on overdue aged main).

Propagation trace (hops from PIPE-ZN-44; `δ^hops`, edge `w`, lag-gated):

child	path	parent impact	w	hops	δ^hops	contrib	lag	active @T+45?
PS-12	PIPE→PS-12	90.0	0.95	1	0.85	72.7	0	yes
R-3	PS-12→R-3	72.7	0.90	2	0.72	47.1	5m	yes
WATER-ZN	R-3→WATER	47.1	1.00	3	0.61	28.9	5m	yes
HOSP-ZN-1	WATER→HOSP	28.9	0.60	4	0.52	9.1	30m	yes
DP-5	WATER→DP-5	28.9	0.70	4	0.52	10.6	25m	yes
JUNC-7	DP-5→JUNC-7	10.6	0.40	5	0.44	1.9	40m	yes
POP-ZN	WATER→POP	28.9	1.00	4	0.52	15.0	0	yes
COMMS-911	POP→COMMS	15.0	0.50	5	0.44	3.3	40m	yes

Final node scores `R(n) = clamp(r(n)+impact)`:

node	r(n) local	+impact	R(n)
WATER-ZN	40	28.9	68.9
HOSP-ZN-1 / CARE-ZN	35	9.1	44.1
POP-ZN	30	15.0	45.0
JUNC-7 / ROAD-ZN	20	1.9	21.9
COMMS-911	25	3.3	28.3

5.5 Multi-level aggregation

Roll up with a **population-weighted blend of mean and max** — never a naive average, which would dilute a severe local failure under many calm nodes:

```
agg(level) = α · max(children R) + (1−α) · Σ(R_i · pop_i) / Σ pop_i        α = 0.4
```

The `max` term preserves the worst hotspot (operationally a governorate is "in crisis" if any sub-service is critical); the pop-weighted mean term reflects breadth of exposure. `α=0.4` weights breadth slightly over the single worst node.

```
def aggregateIndex(level, subject):
    kids = children_at(subject, level−1)
    R = [aggregateIndex(level−1, k) if not leaf(k) else node_score(k) for k in kids]
    pop = [served_pop(k) for k in kids]
    mean_w = sum(r*p for r,p in zip(R,pop)) / max(1,sum(pop))
    return clamp(ALPHA*max(R) + (1−ALPHA)*mean_w, 0, 100)
```

Zarqa governorate roll-up (services in JO-AZ-N, pop weights — POP-ZN size 180000):

service	R	pop
WATER-ZN	68.9	180000
CARE-ZN	44.1	180000
ROAD-ZN	21.9	60000
COMMS-911	28.3	180000

`mean_w = (68.9·180k + 44.1·180k + 21.9·60k + 28.3·180k)/600k = 44.6`; `max = 68.9`. `agg(JO-AZ) = 0.4·68.9 + 0.6·44.6 = 27.56 + 26.76 = 54.3`.

Baseline (pre-rupture, all `r` local only ≈ 30/35/20/25): `agg ≈ 0.4·35 + 0.6·29.5 = 31.7`. **Zarqa index moves 31.7 → 54.3 (Δ +22.6).** National roll-up applies the same operator across governorates; with only Zarqa elevated, national rises modestly (`max`-term keeps it visible, pop-mean keeps it bounded).

5.6 Explainability — factor attribution

Every `+contrib` in §5.3 is logged as an attribution edge so "why the index changed" is **computed**, not narrated. Decompose the Zarqa Δ +22.6 by tracing each child's seed contribution:

```
{ "subject":"JO-AZ", "delta": 22.6, "attribution":[
  {"via":"PIPE-ZN-44→PS-12→R-3→WATER-ZN","points": 13.0,"share":0.57},
  {"via":"…→WATER-ZN→POP-ZN","points": 5.0,"share":0.22},
  {"via":"…→WATER-ZN→HOSP-ZN-1","points": 3.1,"share":0.14},
  {"via":"…→POP-ZN→COMMS-911","points": 0.9,"share":0.04},
  {"via":"…→DP-5→JUNC-7","points": 0.6,"share":0.03} ] }
```

MUST: `Σ attribution.points ≈ delta` (within ε). This ties the index move to PIPE-ZN-44 as the dominant contributor — consistent with §4's root cause, and confirming the 911/hospital surges are downstream symptoms, not drivers.

5.7 Stability, endogenous inputs, anti-gaming

- **Hysteresis/debounce:** level transitions use dual thresholds. A subject enters `CRITICAL` at ≥60 but only exits at ≤52 (4-pt band); a factor must persist ≥2 recompute cycles before changing the published level. Prevents flapping when a signal jitters

around a boundary.

- **Endogenous-input handling:** `ready` and sentiment can move *because* the index moved (feedback). Such inputs are read from the **previous** snapshot only (one-step lag) and damped by `γ`; they MUST NOT be re-injected within the same convergence loop, breaking the self-reference.
- **Anti-gaming:** per-agency signal contribution to `sev` is capped (no single source can drive a node past 0.9 alone); a node needs ≥2 independent corroborating sources (or one telemetry source) to exceed `R=60`. `src_confidence` from chronically-wrong agencies decays (§2), bounding manipulation.

5.8 Recompute triggers

Recompute (incremental over the affected subgraph, not global) on: (a) new/updated Signal observing a scored node; (b) Incident open/close or root-cause attach (§3/§4); (c) edge-weight or readiness update; (d) lag-gate elapse (a pending edge becomes active — scheduled timer). **SHOULD** debounce bursts to ≤1 recompute/5s per governorate. Each recompute emits a new `RiskNode` with `ts` and full attribution for audit.

---

6. Engine Contracts, End-to-End Flow & Acceptance Tests

Purpose: lock the four engines into one composable pipeline with typed contracts, a single Zarqa end-to-end trace, and developer-runnable acceptance tests that use the §0 ground truth as the oracle.

6.1 Engine I/O contracts

Each engine is a pure function over the shared CDG (§1). One line per engine; full schemas live in their own sections.

Engine	`fn(in) → out`	In	Out
§2 Ingest/Resolve	`resolve(raw_signal) → Signal`	agency payload	normalized `Signal` with `observes_ref`, `location_ref` bound to CDG nodes
§3 Correlation	`stitch(Signal[]) → Incident[]`	resolved `Signal[]`	`Incident{signal_ids[], span, primary_location}` + `correlated_with` edges
§4 Root-Cause	`diagnose(Incident) → RootCause`	one `Incident`	`RootCause{cause_node_ref, mechanism, confidence, supporting/conflicting/missing}` + `caused_by` edges
§5 Risk	`score(RootCause, CDG) → RiskNode[]`	`RootCause` (or raw deltas)	`RiskNode[]` (subject, level, score, factors) + index delta
§6 Insight	`surface(Incident, RootCause, RiskNode[]) → Insight`	the three above	duty-officer `Insight` (one card)

The pipeline is `surface ∘ score ∘ diagnose ∘ stitch ∘ map(resolve)`. The platform **MUST** run engines in this order; each engine **MUST** be idempotent on re-run with the same inputs (check: re-invoking produces byte-identical output objects, ts excluded).

```
Insight {
  id, incident_ref, headline, root_cause_ref,
  symptoms[]:        [{signal_ref, observes_ref, role:"symptom"}],
  recommended_focus: node_ref,        # the asset to act on
  suppressed[]:      [node_ref],      # symptom-services NOT to chase
  risk_summary:      {subject, level_before, level_after, drivers[]},
  confidence, provenance              # "live" | "sim"
}
```

6.2 End-to-end Zarqa trace

One row per hop; object shown at each stage. `INC-ZN-WATER` is the stitched incident; `RC-1` the diagnosis.

Hop	Input	Engine	Output object (key fields)
1	`S1` (WAJ SCADA, PS-12, 6.2→1.1 bar)	§2	`Signal S1{observes_ref:PS-12, location_ref:JO-AZ-N, severity:0.85, event_time:T+0}`
1	`S2..S5` likewise	§2	`S2→WATER-ZN`, `S3→HOSP-ZN-1`, `S4→JUNC-7`, `S5→COMMS-911/POP-ZN` all `location_ref:JO-AZ-N`; `S6→national`, topic=fuel
2	`[S1..S6]`	§3	`INC-ZN-WATER{signal_ids:[S1,S2,S3,S4,S5], primary_location:JO-AZ-N, span:{T+0,T+45m}}`; **S6 excluded** (off-topic, national)
3	`INC-ZN-WATER`	§4	`RC-1{cause_node_ref:PIPE-ZN-44, mechanism:"pressure-transient+corrosion", confidence:0.80, supporting:[S1,S2], conflicting:[S5-vol], missing:[inline-leak-sensor]}`
4	`RC-1` + CDG	§5	`RiskNode` cascade (§6.3) → `WATER-ZN` → `CARE-ZN` → `JO-AZ-N` index Δ
5	all above	§6	`Insight INS-1` (§6.4) to duty officer

6.3 Risk cascade (restates §5 — single source of truth)

§5 is authoritative for the cascade; §6 does **not** recompute it on a different scale. Applying §5.3's rule `impact(child) += impact(parent)·w·δ^hops` (δ=0.85) seeded at `r(PIPE-ZN-44)=90`, the §5.4/§5.5 results are (0–100 scale):

Subject	r(n) (§5.4/§5.5)	Level	First active
`WATER-ZN` (water)	**68.9**	high	T+10m
`CARE-ZN` / `HOSP-ZN-1` (health)	**44.1**	moderate	T+40m
`JO-AZ-N` (governorate roll-up, §5.5)	**54.3** (Δ +22.6 from baseline 31.7)	high	T+45m

Ordering water > health (68.9 > 44.1) and onset water (T+10m) < health (T+40m) < governorate (T+45m) match the symptom timing (S1 T+0, S3 T+35m, S4 T+40m). The governorate delta is attributed to `PIPE-ZN-44` (§5.6), not to `PSAP-911` / `HOSP-ZN-1`.

6.4 Surfaced Insight (worked)


```
{ "id":"INS-1", "incident_ref":"INC-ZN-WATER",
  "headline":"Root cause: water trunk-main rupture (PIPE-ZN-44); 911/hospital are symptoms — act on water, not
        ER",
  "root_cause_ref":"RC-1",
  "symptoms":[{"signal_ref":"S5","observes_ref":"COMMS-911","role":"symptom"},
              {"signal_ref":"S3","observes_ref":"HOSP-ZN-1","role":"symptom"}],
  "recommended_focus":"PIPE-ZN-44",
  "suppressed":["COMMS-911","CARE-ZN"],
  "risk_summary":{"subject":"JO-AZ-N","level_before":"low","level_after":"high",
                  "drivers":["PIPE-ZN-44","WATER-ZN"]},
  "confidence":0.80, "provenance":"live" }
```


6.5 Acceptance tests

Developer-runnable Given/When/Then over the §0 fixture. Each is a hard gate (RFC-2119 **MUST**). Oracle = §0 ground truth.

T-CORR — correlation (§3). *Given* resolved `S1..S6`. *When* `stitch()` runs. *Then* exactly one incident `INC-ZN-WATER` **MUST** contain `{S1,S2,S3,S4,S5}` and **MUST NOT** contain `S6`. Check: `assert inc.signal_ids == {S1,S2,S3,S4,S5}` and `S6 ∉ any incident`. Edge: re-ordering signal arrival **MUST** yield the same set.

T-RC — root-cause ranking (§4). *Given* `INC-ZN-WATER`. *When* `diagnose()` ranks candidate cause nodes. *Then* the water-infra node (`PIPE-ZN-44` / `PS-12`) **MUST** rank #1, strictly above `HOSP-ZN-1` and `PSAP-911`; result **MUST** carry `≥2 supporting` (`S1,S2`) and `≥1 conflicting` (high-volume `S5` that does not point upstream). Check: `assert rank[0].node ∈ {PIPE-ZN-44,PS-12} ∧ len(supporting)≥2 ∧ len(conflicting)≥1`.

T-RISK — cascade order & attribution (§5). *Given* baseline CDG. *When* the `PIPE-ZN-44` rupture is injected. *Then* risk **MUST** rise on water before health before governorate (water `R=68.9` ≥ health `R=44.1`; governorate `54.3`, high) with the delta **attributed to `PIPE-ZN-44`**. *And When* the rupture is removed, the index **MUST** revert to baseline (±ε). Check: `assert t(water) < t(health) < t(gov) ∧ delta.driver==PIPE-ZN-44 ∧ abs(index_after_revert − baseline) ≤ ε`.

T-ISO — simulation isolation (§5/§2). *Given* the same rupture with `provenance="sim"`. *When* it is injected. *Then* the **live** governorate index **MUST NOT** move; the insight (if any) **MUST** carry `provenance:"sim"`. Check: `assert live_index == live_index_before ∧ insight.provenance=="sim"`.

All four tests **MUST** pass in CI against the §0 fixture before any engine change merges (check: `pytest tests/zarqa/ -q` exits 0).
